

PRD | AI 应用工程学习 APP

1. 产品名称

AI 应用工程 100 讲

首版可命名为：

AI 应用工程 50 讲

2. 产品定位

这是一个面向企业 AI 应用负责人、AI 平台负责人、研发管理者和高级工程师的学习型 APP。

它不是面向小白的 AI 百科，也不是提示词技巧合集，而是一套帮助用户系统掌握企业级 AI 应用工程的交互式知识系统。

核心定位：

帮助用户把 AI 及其应用相关的关键知识点，从“听过概念”提升到“能解释机制、能判断方案、能诊断问题、能指导落地”。

3. 目标用户

3.1 核心用户

1. 企业 AI 应用负责人
2. MaaS / AI Infra / Agent 平台负责人
3. 企业研发管理者
4. 资深软件工程师
5. AI 转型项目 Owner
6. 希望从“会用 AI 工具”升级到“能设计 AI 系统”的技术人员

3.2 用户画像

用户不是零基础学习者，具备基础软件工程经验，可能了解大模型、RAG、Agent、MaaS、Token、Prompt 等概念，但缺少体系化框架。

他们最关心的不是“概念是什么”，而是：

1. 为什么这个技术有效
2. 工程上怎么落地
3. 出问题怎么定位
4. 业务和组织上怎么评价价值
5. 如何把知识转成平台能力、产品能力和团队能力

4. 产品目标

4.1 首版目标

构建一个可运行的 Web/PWA 学习 APP，覆盖 50 个 AI 应用工程核心知识点，支持：

1. 结构化知识图谱浏览
2. 单个知识点深入学习
3. 动画式机制演示
4. 企业级案例讲解
5. 诊断题训练
6. 学习进度跟踪
7. 收藏和复习
8. 本地数据持久化

4.2 二期目标

扩展到 100 个知识点，增加：

1. 知识点依赖图
2. 学习路径推荐
3. 案例库
4. 诊断题库
5. 术语卡片
6. 个人学习报告
7. AI 辅助问答
8. 内容后台管理

4.3 三期目标

升级为“Agent 工厂实战课”，增加：

1. 交互式实验
2. 代码级演示
3. MaaS 模拟器
4. Agent Loop 沙盘
5. Issue Fix Agent 演练
6. Token 成本计算器
7. RAG 召回实验器
8. 企业 AI 架构设计练习

5. 首版范围

5.1 首版形态

首版做成 Web/PWA，不做原生移动端。

推荐技术栈：

1. React 18
2. Vite

3. TypeScript
4. Zustand
5. React Router
6. CSS Modules 或普通 CSS
7. Framer Motion 或自研轻量动画
8. LocalStorage / IndexedDB
9. Markdown / JSON 内容驱动

5.2 首版不做

1. 不做登录注册
2. 不做服务端
3. 不做付费系统
4. 不做复杂后台
5. 不做多人协作
6. 不做真实 AI 问答
7. 不接大模型 API
8. 不做原生 iOS / Android
9. 不做复杂视频播放器
10. 不做完整题库系统

5.3 首版必须做

1. 首页
2. 专题列表页
3. 知识点详情页
4. 动画演示组件
5. 诊断题组件
6. 进度记录
7. 收藏功能
8. 搜索功能
9. 术语索引
10. 响应式布局

6. 信息架构

APP 分为 6 个一级模块。

6.1 模块一：模型怎么工作

目标：解释大模型的基本机制。

知识点：

1. Token
2. Transformer
3. 注意力机制
4. 上下文窗口
5. 自回归生成
6. 采样策略
7. 指令微调

8. 偏好优化
9. 幻觉
10. 推理能力边界

6.2 模块二：模型怎么跑得又快又稳

目标：解释推理性能、算力效率和系统瓶颈。

知识点：

1. Prefill
2. Decode
3. TTFT
4. TPOT
5. KV Cache
6. P-D 分离
7. Batch 调度
8. 投机解码
9. 量化
10. Roofline

6.3 模块三：模型怎么变成企业平台

目标：解释 MaaS、模型网关、路由、成本、稳定性和治理。

知识点：

1. 模型网关
2. 多模型路由
3. 成本路由
4. 能力路由
5. 限流熔断
6. 计量审计
7. 缓存体系
8. SLA 保障

6.4 模块四：模型怎么变成 Agent

目标：解释 Agent、工具调用、Skill、上下文工程和任务闭环。

知识点：

1. Prompt 与 Context
2. 系统提示词
3. 上下文压缩
4. 上下文污染
5. 分层会话
6. 规格驱动开发
7. AGENTS.md
8. 仓库上下文
9. Agent Loop

10. 工具调用
11. Skill
12. Subagent

6.5 模块五：Agent 怎么改变软件工程

目标：解释 AI 原生软件工程中的需求、问题单、代码、测试、评审和运维闭环。

知识点：

1. Code Review Agent
2. Issue Fix Agent
3. 需求拆解 Agent
4. 测试生成 Agent
5. 运维诊断 Agent
6. 价值复盘 Agent

6.6 模块六：企业怎么治理 AI

目标：解释企业 AI 落地中的评测、安全、成本、组织和 ROI。

知识点：

1. Eval
2. Trace
3. Observability
4. Token ROI
5. 权限治理
6. 人机协同
7. 质量门禁
8. Agent 安全
9. 组织阵型
10. AI 能力模型

7. 页面设计

7.1 首页

目标

让用户快速理解这个 APP 是什么、学什么、怎么学。

页面内容

1. 顶部标题区
2. 产品名：AI 应用工程 50 讲
3. 副标题：从模型原理到 Agent 工厂，把关键知识点学到能落地
4. 学习进度区

5. 已完成知识点数量
6. 总知识点数量
7. 完成比例
8. 连续学习天数
9. 推荐学习路径
10. 入门路径：模型怎么工作
11. 工程路径：推理系统与 MaaS
12. 应用路径：RAG、Agent、Skill
13. 负责人路径：成本、评测、治理、组织
14. 六大模块入口
15. 每个模块显示标题、简介、知识点数量、完成进度
16. 今日推荐
17. 推荐 1 个未学习知识点
18. 推荐 1 个复习知识点
19. 推荐 1 道诊断题

交互

1. 点击模块进入专题列表
2. 点击知识点进入详情页
3. 点击继续学习进入上次学习位置
4. 点击搜索进入搜索页

7.2 专题列表页

目标

展示某个模块下的全部知识点，并呈现学习路径。

页面内容

1. 模块标题
2. 模块简介
3. 学习路径说明
4. 知识点列表
5. 每个知识点卡片显示：
6. 标题

7. 一句话定义
8. 难度
9. 预计学习时长
10. 是否有动画
11. 是否完成
12. 是否收藏

交互

1. 点击知识点进入详情页
2. 点击收藏按钮切换收藏状态
3. 点击筛选按钮按难度、状态、是否有动画筛选
4. 点击排序按钮按推荐顺序、学习时长、难度排序

7.3 知识点详情页

目标

让用户把一个知识点从概念、机制、案例、诊断题四层吃透。

页面结构

1. 标题区
2. 一句话定义
3. 为什么重要
4. 机制讲解
5. 动画演示
6. 企业案例
7. 常见误区
8. 诊断题
9. 总结卡片
10. 关联知识点
11. 完成学习按钮

标准内容结构

每个知识点都使用统一结构：

1. definition：一句话定义
2. whyItMatters：为什么重要
3. mentalModel：心智模型
4. mechanism：机制讲解
5. animation：动画配置
6. enterpriseCase：企业案例
7. pitfalls：常见误区
8. diagnosticQuestion：诊断题
9. keyTakeaways：核心结论
10. relatedConcepts：关联知识点

7.4 动画演示页 / 组件

目标

用简单动画解释抽象机制。

首版不做复杂 3D，不做视频。全部使用 SVG、CSS、Canvas 或 React 组件实现。

首版必须支持的动画类型

1. TokenFlowAnimation
2. 展示文本如何被切成 Token
3. 展示 Token 如何逐个进入模型
4. 展示模型如何逐个输出 Token
5. AttentionAnimation
6. 展示一个词如何关注上下文中的其他词
7. 用线条粗细表示注意力强弱
8. ContextWindowAnimation
9. 展示上下文窗口容量
10. 展示新消息进入、旧消息被压缩或淘汰
11. KVCacheAnimation
12. 展示 Prefill 阶段写入 KV Cache
13. 展示 Decode 阶段复用 KV Cache
14. 展示命中和不命中的区别
15. PrefillDecodeAnimation
16. 展示 Prefill 与 Decode 两个阶段
17. 展示 TTFT 与 TPOT 的位置
18. RAGPipelineAnimation
19. 展示 query -> rewrite -> retrieve -> rerank -> generate -> cite
20. AgentLoopAnimation

21. 展示 observe -> think -> act -> check -> continue / stop

22. ModelRoutingAnimation

23. 展示请求如何按成本、能力、延迟路由到不同模型

动画交互要求

1. 支持播放 / 暂停
2. 支持上一步 / 下一步
3. 支持重置
4. 每一步有简短解释
5. 移动端可用
6. 不依赖远程资源

7.5 诊断题页面 / 组件

目标

通过企业真实场景训练用户的工程判断能力。

诊断题结构

1. 场景描述
2. 现象
3. 可选判断
4. 用户选择
5. 解析
6. 推荐排查路径
7. 关联知识点

题型

首版只做单选和多选。

示例

题目：

某 MaaS 平台在高峰期 TTFT 从 800ms 上升到 4s，KV Cache 命中率从 60% 下降到 15%，用户反馈首字等待明显变长。最优先排查什么？

选项：

- A. 模型参数量是否过大
- B. Session 亲和是否失效
- C. 温度参数是否过高
- D. 是否需要增加提示词长度

正确答案：

B

解析：

TTFT 上升且 KV Cache 命中率下降，优先怀疑请求没有路由到持有上下文缓存的实例，或者会话亲和策略失效。模型大小和温度可能影响整体质量或速度，但不是这个现象的首要解释。

7.6 搜索页

目标

让用户快速找到知识点、术语、案例和诊断题。

搜索范围

1. 知识点标题
2. 一句话定义
3. 机制讲解
4. 企业案例
5. 常见误区
6. 诊断题

交互

1. 输入关键词实时搜索
 2. 搜索结果按类型分组
 3. 支持点击跳转
 4. 支持空结果提示
 5. 支持最近搜索记录
-

7.7 术语索引页

目标

提供 AI 应用工程术语字典。

内容

每个术语包含：

1. 中文名
2. 英文名
3. 一句话解释
4. 所属模块
5. 关联知识点

示例

术语: TTFT

中文: 首 Token 时间

解释: 从请求发出到模型返回第一个 Token 的时间, 主要受排队、Prefill、调度和缓存命中影响。

关联知识点: Prefill、KV Cache、Batch 调度、SLA 保障。

7.8 我的学习页

目标

展示个人学习进度。

内容

1. 总完成进度
 2. 各模块完成进度
 3. 最近学习
 4. 收藏知识点
 5. 错题记录
 6. 推荐复习
-

8. 数据模型

8.1 KnowledgePoint

```
export interface KnowledgePoint {  
  id: string;  
  title: string;  
  slug: string;  
  moduleId: string;  
  order: number;  
  difficulty: 'basic' | 'intermediate' | 'advanced';  
  estimatedMinutes: number;  
  tags: string[];  
  hasAnimation: boolean;  
  definition: string;  
  whyItMatters: string;  
  mentalModel: string;  
  mechanism: string[];  
  animation?: AnimationConfig;  
  enterpriseCase: EnterpriseCase;  
  pitfalls: string[];  
  diagnosticQuestion?: DiagnosticQuestion;  
  keyTakeaways: string[];  
  relatedConceptIds: string[];  
}
```

8.2 LearningModule

```
export interface LearningModule {  
  id: string;  
  title: string;  
  subtitle: string;  
  description: string;  
  order: number;  
  conceptIds: string[];  
  recommendedFor: string[];  
}
```

8.3 AnimationConfig

```
export interface AnimationConfig {  
  type:  
    | 'token-flow'  
    | 'attention'  
    | 'context-window'  
    | 'kv-cache'  
    | 'prefill-decode'  
    | 'rag-pipeline'  
    | 'agent-loop'  
    | 'model-routing';  
  title: string;  
  steps: AnimationStep[];  
}  
  
export interface AnimationStep {  
  id: string;  
  title: string;  
  description: string;  
  highlightTargets?: string[];  
  durationMs?: number;  
}
```

8.4 EnterpriseCase

```
export interface EnterpriseCase {  
  title: string;  
  scenario: string;  
  problem: string;  
  analysis: string;  
  solution: string;  
  takeaway: string;  
}
```

8.5 DiagnosticQuestion

```
export interface DiagnosticQuestion {  
  id: string;  
  type: 'single' | 'multiple';  
  scenario: string;  
  question: string;  
  options: DiagnosticOption[];  
  correctOptionIds: string[];  
  explanation: string;  
  troubleshootingPath: string[];  
  relatedConceptIds: string[];  
}  
  
export interface DiagnosticOption {  
  id: string;  
  text: string;  
}
```

8.6 UserProgress

```
export interface UserProgress {  
  completedConceptIds: string[];  
  favoriteConceptIds: string[];  
  wrongQuestionIds: string[];  
  lastVisitedConceptId?: string;  
  lastStudyDate?: string;  
  studyStreakDays: number;  
}
```

9. 推荐目录结构

```
src/  
  app/  
    App.tsx  
    router.tsx  
  pages/  
    HomePage.tsx  
    ModulePage.tsx  
    ConceptPage.tsx  
    SearchPage.tsx  
    GlossaryPage.tsx  
    ProfilePage.tsx  
  components/  
    layout/  
      AppShell.tsx
```

```
Header.tsx
Sidebar.tsx
BottomNav.tsx
concept/
  ConceptCard.tsx
  ConceptHeader.tsx
  ConceptSection.tsx
  TakeawayBox.tsx
  RelatedConcepts.tsx
animation/
  AnimationPlayer.tsx
  TokenFlowAnimation.tsx
  AttentionAnimation.tsx
  ContextWindowAnimation.tsx
  KVCacheAnimation.tsx
  PrefillDecodeAnimation.tsx
  RAGPipelineAnimation.tsx
  AgentLoopAnimation.tsx
  ModelRoutingAnimation.tsx
quiz/
  DiagnosticQuestion.tsx
  OptionCard.tsx
  ExplanationPanel.tsx
progress/
  ProgressBar.tsx
  ModuleProgress.tsx
  StudyStats.tsx
search/
  SearchBox.tsx
  SearchResults.tsx
data/
  modules.ts
  concepts.ts
  glossary.ts
store/
  progressStore.ts
types/
  index.ts
utils/
  search.ts
  progress.ts
styles/
  tokens.css
  global.css
```

10. 路由设计

/	首页
/modules	模块总览
/modules/:moduleId	模块详情
/concepts/:slug	知识点详情
/search	搜索
/glossary	术语索引
/profile	我的学习

11. 核心组件要求

11.1 AppShell

负责整体布局。

要求：

1. 桌面端左侧导航
2. 移动端底部导航
3. 顶部搜索入口
4. 主内容区最大宽度限制
5. 保持页面简洁

11.2 ConceptCard

用于模块页展示知识点。

展示字段：

1. 标题
2. 一句话定义
3. 难度
4. 学习时长
5. 是否完成
6. 是否收藏
7. 是否有动画

11.3 AnimationPlayer

统一动画播放器。

功能：

1. 播放
2. 暂停
3. 下一步
4. 上一步

5. 重置
6. 显示当前步骤说明
7. 支持不同动画类型注册

11.4 DiagnosticQuestion

诊断题组件。

功能：

1. 渲染题干
2. 渲染选项
3. 支持单选 / 多选
4. 提交答案
5. 显示正确 / 错误
6. 展示解析
7. 记录错题

11.5 ProgressStore

使用 Zustand 管理学习进度。

功能：

1. 标记完成
2. 取消完成
3. 收藏
4. 取消收藏
5. 记录错题
6. 记录最近访问
7. 计算模块完成度
8. 持久化到 LocalStorage

12. 视觉规范

12.1 风格

采用“华为红·咨询式技术汇报”风格。

核心原则：

1. 白底
2. 大留白
3. 强网格
4. 少边框
5. 小圆角或直角
6. 信息密度高但层级清晰
7. 一屏一个核心结论
8. 不使用花哨渐变
9. 不使用大面积深色块

10. 不使用阴影卡片堆砌

12.2 色彩

```
:root {  
  --color-bg: #ffffff;  
  --color-title: #111111;  
  --color-text: #3f3f3f;  
  --color-muted: #777777;  
  --color-border: #d9d9d9;  
  --color-soft-bg: #f5f6f7;  
  --color-red: #c7000b;  
  --color-red-soft: #fff3f3;  
}
```

12.3 强调规则

只允许三种强调方式：

1. 华为红
2. 字重
3. 位置

不要同时使用阴影、粗边框、底色、大圆角等多种方式强调。

12.4 字体

优先使用系统字体。

```
font-family:  
  -apple-system,  
  BlinkMacSystemFont,  
  "Segoe UI",  
  "PingFang SC",  
  "Microsoft YaHei",  
  sans-serif;
```

12.5 布局

1. 桌面端最大内容宽度 1200px
2. 详情页正文最大宽度 860px
3. 模块卡片使用 2 或 3 列网格
4. 移动端单列
5. 卡片间距 16px 或 24px
6. 页面上下留白 32px 到 64px

13. 内容样例

首版至少内置 12 个高质量知识点，用于验证产品体验。

13.1 Token

一句话定义：

Token 是模型处理文本的最小输入输出单位，不一定等于一个字或一个词。

为什么重要：

Token 决定上下文长度、计费、推理成本和生成速度。企业 AI 应用中的成本、时延和上下文治理，最终都会落到 Token 规模上。

心智模型：

把 Token 理解为模型看到的“文字积木”。用户看到的是自然语言，模型看到的是一串 Token 编号。

机制讲解：

1. 文本先经过分词器转换为 Token
2. 每个 Token 对应一个整数编号
3. 模型把 Token 编号转换为向量
4. 模型基于上下文预测下一个 Token
5. 输出 Token 再被解码为人类可读文本

企业案例：

一个 Agent 应用每轮都把完整历史、工具结果和大段文档塞进上下文，导致输入 Token 激增。用户感知是响应变慢，平台侧表现为成本上升、TTFT 变长、缓存命中下降。

常见误区：

1. 以为一个汉字就是一个 Token
2. 只关注输出 Token，不关注输入 Token
3. 认为上下文越长越好
4. 忽略工具结果带来的 Token 膨胀

核心结论：

1. Token 是 AI 应用成本和性能的基础计量单位
2. 上下文治理首先是 Token 治理
3. 企业平台必须观测输入 Token、输出 Token 和缓存命中率

13.2 KV Cache

一句话定义：

KV Cache 是在大模型推理中缓存历史上下文的 Key 和 Value，避免每次生成新 Token 时重复计算全部历史。

为什么重要：

KV Cache 直接影响首字时延、吞吐、显存占用和多轮对话体验。企业 MaaS 平台如果 KV Cache 命中率低，高并发下会快速劣化。

心智模型：

把 KV Cache 理解为“模型已经读过的上下文笔记”。如果下一轮还在同一个会话里，模型可以直接翻笔记，而不是重新读全文。

机制讲解：

1. Prefill 阶段处理完整输入上下文
2. 模型为每层注意力生成 Key 和 Value
3. 这些 Key 和 Value 被缓存到显存中
4. Decode 阶段生成新 Token 时复用已有 KV
5. 如果请求被路由到没有缓存的实例，就需要重新 Prefill

企业案例：

某 MaaS 平台高峰期 TTFT 明显上升。排查发现，负载均衡策略没有保持 Session 亲和，多轮请求被分配到不同实例，导致 KV Cache 命中率下降。

常见误区：

1. 只关注模型大小，不关注缓存命中
2. 认为增加卡数一定能降低 TTFT
3. 忽视会话亲和
4. 忽视长上下文对显存的挤占

诊断题：

某平台 TTFT 从 800ms 上升到 4s，同时 KV Cache 命中率从 60% 下降到 15%。最优先排查什么？

正确方向：

排查 Session 亲和、路由策略、实例缓存状态和请求是否被错误打散。

核心结论：

1. KV Cache 是多轮对话性能的关键
2. Session 亲和是企业 MaaS 必须具备的能力
3. 高并发问题不能只看平均时延，要同时看缓存命中率

13.3 Agent Loop

一句话定义：

Agent Loop 是 Agent 围绕目标不断观察、思考、行动、校验并决定是否继续的执行循环。

为什么重要：

Chatbot 只回答问题，Agent 要完成任务。二者的分水岭就是是否有稳定的循环执行、工具调用、状态管理和退出机制。

心智模型：

把 Agent Loop 理解为一个带工具的工程师：它不是一次回答完，而是一边看情况、一边做动作、一边检查结果，直到任务完成或触发退出条件。

机制讲解：

1. 用户给出目标
2. Agent 读取上下文
3. Agent 生成计划
4. Agent 调用工具
5. Agent 观察工具结果
6. Agent 判断是否继续
7. 成功则输出结果
8. 失败则重试、回滚或转人工

企业案例：

Issue Fix Agent 接到问题单后，不应该直接改代码。它需要先理解问题单、补全复现路径、定位相关文件、制定修改计划、执行修改、运行测试、生成 PR，并在人审前给出风险说明。

常见误区：

1. 把 Agent 当成更长的 Prompt
2. 只设计执行，不设计退出
3. 只关注自动化，不关注回滚
4. 缺少人工审批点
5. 缺少过程 Trace

核心结论：

1. Agent 的关键不是会说，而是会循环执行
2. Agent Loop 必须有状态、工具、校验和退出条件
3. 企业 Agent 必须可追踪、可回滚、可审批

14. 本地持久化

首版所有用户状态保存在 LocalStorage。

Key 设计：

```
const STORAGE_KEY = 'ai-learning-app-progress-v1';
```

保存内容：

1. 已完成知识点
2. 收藏知识点
3. 错题记录
4. 最近访问
5. 学习天数

要求：

1. 页面刷新后进度不丢失
2. 支持清空学习记录
3. 数据结构要有 version 字段，方便未来迁移

15. 搜索实现

首版使用前端本地搜索。

搜索字段：

1. title
2. definition
3. tags
4. mechanism
5. enterpriseCase
6. pitfalls

排序规则：

1. 标题完全匹配优先
2. 标题包含关键词次之
3. 标签匹配再次
4. 正文匹配最后

搜索结果类型：

1. 知识点
2. 术语
3. 诊断题

16. 验收标准

16.1 功能验收

1. 用户可以浏览 6 个模块

2. 用户可以进入任意知识点详情页
3. 用户可以播放至少 4 类动画
4. 用户可以完成诊断题并查看解析
5. 用户可以标记知识点完成
6. 用户可以收藏知识点
7. 用户刷新页面后进度不丢失
8. 用户可以搜索知识点
9. 用户可以查看术语索引
10. 移动端和桌面端均可正常使用

16.2 内容验收

1. 首版至少内置 12 个完整知识点
2. 每个知识点必须包含一句话定义
3. 每个知识点必须包含机制讲解
4. 每个知识点必须包含企业案例
5. 每个知识点必须包含常见误区
6. 至少 8 个知识点包含诊断题
7. 至少 6 个知识点包含动画配置

16.3 视觉验收

1. 整体白底
2. 使用华为红作为唯一强调色
3. 不使用渐变
4. 不使用大面积深色背景
5. 不使用卡片阴影堆砌
6. 页面层级清晰
7. 移动端没有横向滚动
8. 内容区阅读舒适

16.4 工程验收

1. TypeScript 无类型错误
2. ESLint 无严重错误
3. 构建成功
4. 路由刷新不报错
5. 组件拆分清晰
6. 数据和组件分离
7. 动画组件可复用
8. 状态管理集中
9. 无未使用的大型依赖
10. README 说明如何启动项目

17. 开发里程碑

17.1 第 0 阶段：项目初始化

目标：

创建可运行项目骨架。

任务：

1. 初始化 Vite + React + TypeScript
2. 配置路由
3. 配置全局样式
4. 建立目录结构
5. 创建基础 Layout
6. 创建 mock 数据

验收：

1. `npm install` 成功
 2. `npm run dev` 成功
 3. 首页可访问
 4. 路由可跳转
-

17.2 第 1 阶段：首页和模块页

目标：

完成基本浏览体验。

任务：

1. 实现首页
2. 实现模块卡片
3. 实现模块详情页
4. 实现知识点卡片
5. 接入基础数据

验收：

1. 能看到 6 个模块
 2. 能进入模块页
 3. 能看到知识点列表
 4. 卡片展示状态正确
-

17.3 第 2 阶段：知识点详情页

目标：

完成核心学习体验。

任务：

1. 实现知识点详情页

2. 渲染定义、机制、案例、误区、总结
3. 实现关联知识点
4. 实现完成按钮
5. 实现收藏按钮

验收：

1. 任意知识点可打开
 2. 内容结构完整
 3. 完成状态可保存
 4. 收藏状态可保存
-

17.4 第 3 阶段：动画组件

目标：

完成核心可视化能力。

任务：

1. 实现 AnimationPlayer
2. 实现 TokenFlowAnimation
3. 实现 KVCacheAnimation
4. 实现 AgentLoopAnimation
5. 实现 RAGPipelineAnimation

验收：

1. 动画可以播放
 2. 可以暂停
 3. 可以切换步骤
 4. 步骤说明同步变化
 5. 移动端可用
-

17.5 第 4 阶段：诊断题

目标：

完成训练闭环。

任务：

1. 实现诊断题组件
2. 支持单选
3. 支持多选
4. 支持提交答案
5. 支持解析展示
6. 支持错题记录

验收：

1. 答题流程顺畅
2. 正确错误判断准确
3. 解析展示清晰
4. 错题可记录

17.6 第 5 阶段：搜索、术语、我的学习

目标：

完成 APP 闭环。

任务：

1. 实现搜索页
2. 实现术语索引页
3. 实现我的学习页
4. 实现进度统计
5. 实现最近学习
6. 实现清空记录

验收：

1. 搜索可用
2. 术语可浏览
3. 学习进度准确
4. 最近学习准确
5. 清空记录可用

17.7 第 6 阶段：打磨与发布

目标：

达到可演示状态。

任务：

1. 响应式适配
2. 视觉统一
3. 内容补齐
4. 构建检查
5. README 编写
6. 部署配置

验收：

1. 桌面端体验稳定

2. 移动端体验稳定
 3. 构建成功
 4. README 完整
 5. 可部署到静态站点
-

18. 推荐给 Codex 的开发方式

18.1 第一轮任务

请先完成项目初始化和基础骨架，不要一次性实现所有功能。

任务：

1. 创建 Vite + React + TypeScript 项目
2. 建立推荐目录结构
3. 配置 React Router
4. 创建全局视觉变量
5. 创建 mock 数据
6. 实现首页和模块页
7. 保证项目可运行

18.2 第二轮任务

完成知识点详情页和本地进度管理。

任务：

1. 实现 ConceptPage
2. 实现 Zustand progressStore
3. 实现完成、收藏、最近访问
4. 数据持久化到 LocalStorage
5. 补充 12 个知识点样例

18.3 第三轮任务

完成动画和诊断题。

任务：

1. 实现 AnimationPlayer
2. 实现 4 个动画组件
3. 实现 DiagnosticQuestion
4. 接入知识点详情页
5. 保证动画和题目配置由数据驱动

18.4 第四轮任务

完成搜索、术语和我的学习页。

任务：

1. 实现本地搜索
2. 实现术语索引
3. 实现学习进度页
4. 完成移动端适配
5. 完成 README

19. Codex 执行约束

1. 不要引入复杂后端
2. 不要过早引入数据库
3. 不要接入真实大模型 API
4. 不要引入重型 UI 框架
5. 不要使用花哨视觉风格
6. 不要生成大面积深色主题
7. 不要使用渐变和阴影堆砌
8. 不要把内容写死在组件里
9. 所有知识点内容必须数据化
10. 所有动画步骤必须配置化
11. 每完成一个阶段都要保证项目可运行
12. 代码优先保证清晰和可维护，不追求炫技

20. 成功标准

首版成功的标准不是“内容很多”，而是：

1. 用户打开后能立刻理解这是 AI 应用工程学习系统
2. 用户能沿着 6 个模块形成完整知识地图
3. 用户能通过动画看懂复杂机制
4. 用户能通过诊断题训练工程判断
5. 用户能记录学习进度
6. 项目结构清晰，后续容易扩展到 100 个知识点
7. 内容、动画、题目都能通过数据配置持续扩展

最终交付物：

1. 一个可运行的 Web/PWA 学习 APP
2. 覆盖 50 个知识点的信息架构
3. 首版内置至少 12 个完整知识点
4. 至少 4 类动画演示
5. 至少 8 道诊断题
6. 完整 README
7. 可静态部署的构建产物